Entrega G2 — Resposta à rubrica G1

Branch: `fix/meus-pedidos-isolamento-usuario`
PR: [abra link após push]

Este documento descreve o que foi implementado para endereçar cada ponto da rubrica da Entrega G1 (nota inicial: **45/100**). Rubrica original em `2026-01-ES - G1-Obra Fácil.pdf` e gap analysis em `docs/gap-analysis-entrega-g1.md`.

RF1 — Fluxo de negócio (cliente) — estava 0,3/18

> *Observação da banca: "Histórico traz dados de outros usuários. Necessário ver 'meus pedidos'."*

Root cause: `ClerkAuthGuard` em modo bypass (`DISABLE_CLERK_AUTH=true`) executava `SELECT * FROM profiles LIMIT 1` sem `ORDER BY`. Todos os "usuários logados" colapsavam no mesmo perfil arbitrário.

Correções aplicadas:

1. **Guard determinístico** (`apps/backend/src/core/guards/clerk-auth.guard.ts`):
   - Bypass respeita header `X-Dev-User-Id` (clerk_id do seed) quando presente.
   - Fallback determinístico: `SELECT ... WHERE role='client' ORDER BY id ASC LIMIT 1`.
   - Em modo Clerk real, faz **JIT provisioning** via `INSERT ... ON CONFLICT` se o profile ainda não existe — idempotente com o webhook.
   - Header X-Dev-User-Id é **ignorado em produção** (verificação dupla via `process.env.DISABLE_CLERK_AUTH`).
2. **Startup guard anti-misconfiguração** (`main.ts`): se `NODE_ENV=production && DISABLE_CLERK_AUTH=true`, backend aborta com `process.exit(1)`.
3. **`.env.example` default seguro**: `DISABLE_CLERK_AUTH=false` (era `true`).
4. **Error message sanitizada**: retorna `"Não autorizado"` genérico (não vaza clerk_id).
5. **Frontend propaga header** (`apps/frontend/src/lib/api/client.ts` + `auth-bypass.ts`): injeta `X-Dev-User-Id` em todas as chamadas quando bypass ativo.
6. **Constante compartilhada** `DEV_USER_ID_HEADER` em `@obrafacil/shared/constants.ts` — uma única fonte de verdade.

Testes:

- Unit (17 cenários): `apps/backend/src/core/guards/clerk-auth.guard.spec.ts`
- E2e isolamento (5 cenários): `apps/backend/test/orders.e2e-spec.ts`
- Playwright: `apps/frontend/tests-e2e/cliente-meus-pedidos.spec.ts`

RF2 — Fluxo de negócio (profissional) — estava 0/18

> *Observação da banca: "Pendente disponibilizar credenciais com perfil do profissional."*

Implementado:

Leitura — dashboard

1. **`GET /api/v1/professionals/me/dashboard`**
   (`apps/backend/src/modules/professionals/professionals.controller.ts`) retorna:
   - Stats agregadas: visitas agendadas, obras ativas, conversas pendentes, obras concluídas.
   - Lista de próximas visitas confirmadas (limit 10).
   - Lista de obras `scheduled + active` com progresso.
   - Protegido por `role === 'professional'` (403 para outros roles).
2. **Página frontend** `/profissional/dashboard`
   (`apps/frontend/src/app/(app)/profissional/dashboard/page.tsx`) — server component que consome o endpoint e renderiza cards + listas com ações.

Ações — fluxo ponta-a-ponta do profissional

Todos os endpoints abaixo exigem `role=professional` **e** que o recurso pertença ao profissional logado (via `professional_id` + `findByProfileId`).

Obras (`apps/backend/src/modules/works/works.controller.ts`):

Endpoint	Transição	Efeito lateral
`PATCH /v1/works/:id/start`	`scheduled` → `active`	define `started_at=now()`
`PATCH /v1/works/:id/complete`	`active` → `completed`	define `completed_at=now()` , `progress_pct=100`
`PATCH /v1/works/:id/progress`	qualquer → mesma	atualiza `progress_pct` (0-100 validado)

Visitas (`apps/backend/src/modules/visits/visits.controller.ts` , já existiam):

Endpoint	Transição	Quem pode
`PATCH /v1/visits/:id/cancel`	`confirmed` → `cancelled`	cliente ou profissional
`PATCH /v1/visits/:id/complete`	`confirmed` → `completed`	apenas profissional

UI frontend (`dashboard/WorkActions.tsx`): botões contextuais por status, usa `router.refresh()` após sucesso. VisitActions com "Concluir"/"Cancelar" em visitas confirmadas. WorkActions com "Iniciar obra"/"Marcar como concluída" conforme status.

Segurança das ações

Guard compartilhado `assertIsWorksProfessional` :

1. Obra existe (404 se não).
2. `profile.role === 'professional'` (403 se não).
3. `professional.id` resolvido via `findByProfileId(profile.id)` bate com `work.professional_id` (403 se não).

Fix lateral: `WorksController.findAll` tinha bug pré-existente — passava `profile.id` direto para `findAllByProfessional` que espera `professional.id` . Corrigido via lookup no repositório de professionals.

Credenciais de teste (docker/02-seed.sql)

clerk_id	Nome	Role
`demo_client_001` (default bypass)	Carlos Alberto	client
`demo_client_002`	Joana Mendes	client
`demo_professional_001`	Ricardo Silva	professional
`demo_professional_002`	José da Silva	professional
`demo_professional_003`	Ana Rodrigues	professional

Para virar profissional no frontend: setar `NEXT_PUBLIC_BYPASS_USER_CLERK_ID=demo_professional_001` em `apps/frontend/.env.local` e reiniciar.

Testes

- Unit backend — `visits.service.spec.ts` (13 cenários): router findAll, book (happy/constraint/zod), cancel (3 paths), complete (3 paths), getAvailableSlots (empty/booked).
- Unit backend — `works.repository.spec.ts` (6 cenários): filter por client/professional, findById (found/missing), updateProgress.
- E2e Playwright — `profissional-dashboard.spec.ts` (condicional via `E2E_BYPASS_USER`).

RF3 — Tecnologia de fronteira (IA) — estava 0/12

Implementado: integração **real e verificável** com **Anthropic Claude (Haiku 4.5)**.

1. **Service** `apps/backend/src/modules/ai/ai.service.ts`:
   - Usa `@anthropic-ai/sdk` oficial.
   - Prompt sistêmico em português, especialista em materiais de construção BR.
   - Schema de resposta: lista de itens `(name, quantity, unit, category)`, estimativa total, notas.
   - Extração robusta de JSON (aceita fences ```json).
   - Logs estruturados com tokens usados.
2. **Controller** `apps/backend/src/modules/ai/ai.controller.ts`:
   - `POST /api/v1/ai/material-quote` com body `{ description: string }`.
   - Validação: 10 ≤ len(description) ≤ 2000.
   - Retorna 500 com mensagem clara se `ANTHROPIC_API_KEY` não configurada.
3. **Página frontend** `/cotacao/ia` (`apps/frontend/src/app/(app)/cotacao/ia/page.tsx`):
   - Client component com textarea + contador de caracteres.
   - Loading state + error handling.
   - Renderiza resultado com itens agrupados, estimativa total, observações.

Validação prática: `curl -X POST ... -d '{"description":"Reformar banheiro pequeno de 4m²"}'` retorna 23 materiais realistas em JSON, com notas contextuais sobre impermeabilização, dimensões etc.

Testes: `apps/backend/src/modules/ai/ai.service.spec.ts` (5 cenários — mock do SDK, parsing, error paths).

RNF-04 — Observabilidade e Rastreabilidade — estava 0/4

Implementado:

1. **Logger estruturado**: `nestjs-pino` configurado em `apps/backend/src/app.module.ts`:
   - Saída JSON em produção, pretty em dev.
   - **Request ID** automático (lê `x-request-id` / `x-correlation-id` ou gera UUID).
   - **Redact** de campos sensíveis: `authorization`, `cookie`, `x-dev-user-id`.
   - Level configurável via `LOG_LEVEL`.
2. **Endpoint de health check**: `GET /api/health` em `apps/backend/src/modules/health/health.controller.ts`:
   - Testa conexão com DB (`SELECT 1`).
   - Retorna `status`, `db`, `uptime_s`, `response_ms`, `version`, `env`, `timestamp`.
   - Status `degraded` se DB estiver down.

RNF-05 — Manutenibilidade e Testabilidade — estava 0/4

Antes: único teste era o placeholder "Hello World".

Agora — 68 testes passando:

Camada	Framework	Testes	Arquivos
Backend unit	Jest	49	`clerk-auth.guard.spec.ts` (17), `orders.service.spec.ts` (5), `ai.service.spec.ts` (5), `visits.service.spec.ts` (13), `works.repository.spec.ts` (6), `app.controller.spec.ts` (3)
Backend e2e	Jest + supertest	6	`orders.e2e-spec.ts` (5 isolamento), `app.e2e-spec.ts` (1 health)
Frontend unit	Vitest + Testing Library	9	`StatusBadge.test.tsx` (4), `PedidosTabs.test.tsx` (5)

Camada	Framework	Testes	Arquivos
Frontend e2e	Playwright	4	`cliente-meus-pedidos.spec.ts` (3), `profissional-dashboard.spec.ts` (1 condicional)

CI (`.github/workflows/ci.yml`):

- Sobe container Postgres com schema + seed aplicados.
- Roda: lint → typecheck → unit tests → e2e backend → build → e2e Playwright.
- Auditoria de segurança no final.

RNF-06 — Portabilidade e Implantação — estava 0,8/4

> *Observação da banca: "Artefatos IaC não identificados."*

Implementado:

1. **Containers OCI** (já existia): `apps/backend/Dockerfile` , `apps/frontend/Dockerfile` , `docker-compose.yml` .
2. **Schema SQL versionado** (já existia): `docker/01-schema.sql` , `docker/02-seed.sql` .
3. **Terraform** (NOVO) em `infra/terraform/vercel/` :
   - `main.tf` : declara os dois projetos Vercel (backend + frontend), env vars, integração com GitHub.
   - `variables.tf` : inputs com `sensitive = true` para tokens/secrets.
   - `terraform.tfvars.example` : template de config.
   - `README.md` : instruções de apply.
   - `.gitignore` atualizado para nunca commitar `*.tfstate` e `*.tfvars` reais.

Decisão: Terraform só para Vercel (não Supabase). O provider do Supabase gerencia apenas projeto/branches — schema e RLS já estão versionados em SQL.

Outros pontos que já estavam OK

- RNF-01 Acessibilidade: Next.js 15 responsivo, HTML5/CSS3/ES2020+.
- RNF-02 Segurança: Clerk + HTTPS Vercel. Reforçado com: startup guard, error sanitization, header validation.

- RNF-03 Interoperabilidade: API REST em `/api/v1/*` com Swagger em `/api/docs`.
- RNF-07 Persistência: PostgreSQL (Supabase).
- RNF-08 Governança: npm workspaces, `.env.example`, Conventional Commits.

Como validar localmente

```
# Subir banco local
docker compose up -d db

# Terminal 1: backend (porta 3333)
cd apps/backend && cp .env.example .env
# editar .env com ANTHROPIC_API_KEY=sk-ant-... para testar IA
DOTENV_CONFIG_PATH=apps/backend/.env node --require dotenv/config apps/backend/dist/main.js

# Terminal 2: frontend (porta 3000)
NEXT_PUBLIC_DISABLE_CLERK_AUTH=true \
  NEXT_PUBLIC_API_URL=http://localhost:3333/api \
  npm run dev:frontend

# Testes
npm test --workspace=backend               # 49 unit (Jest)
npm run test:e2e --workspace=backend       # 6 e2e (supertest + Postgres)
npm test --workspace=frontend              # 9 unit (Vitest)
npm run test:e2e --workspace=frontend      # 4 Playwright
```

Navegar em http://localhost:3000:

- `/pedidos` — Meus Pedidos (isolados por cliente)
- `/cotacao/ia` — Gerar lista de materiais via Claude
- `/profissional/dashboard` — (setar `NEXT_PUBLIC_BYPASS_USER_CLERK_ID=demo_professional_001` primeiro) — agora com botões "Iniciar obra", "Concluir", "Cancelar visita"

Swagger: http://localhost:3333/api/docs
Health: http://localhost:3333/api/health

Variáveis obrigatórias em produção (Vercel)

Backend (`app-devai-backend`):

- `DATABASE_URL` — pooler do Supabase
- `CLERK_SECRET_KEY` , `CLERK_WEBHOOK_SECRET`
- `ANTHROPIC_API_KEY` — necessária para `/v1/ai/material-quote`
- `CORS_ORIGIN`
- `NODE_ENV=production` (ativa o startup guard que impede `DISABLE_CLERK_AUTH=true` em prod)

Frontend (`app-devai-frontend`):

- `NEXT_PUBLIC_API_URL`
- `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` , `CLERK_SECRET_KEY`

Resumo quantitativo

Antes (G1)	Depois (G2)
45/100 pontos	Potencial ~100/100
1 teste (Hello World)	**68 testes** em 4 frameworks
Logs `console.log`	Pino JSON + requestId + redact
Sem IaC	Terraform + Docker + SQL versionados
Sem IA	Claude Haiku 4.5 integrado
Bug crítico em Meus Pedidos	Corrigido + testes de isolamento
Sem dashboard profissional	Dashboard com stats + **ações ponta-a-ponta** (iniciar/concluir obra, concluir/cancelar visita)